



Push and Learn



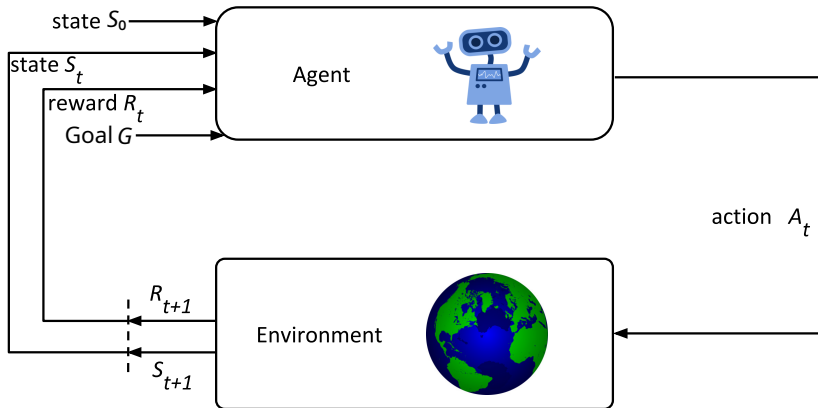
A Hands-on Reinforcement Learning Tutorial
on Dense and Sparse Rewards

Sayantan Auddy

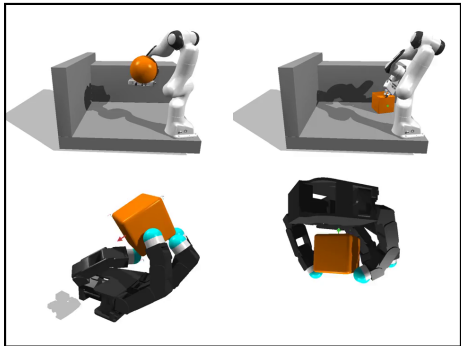
TU Berlin

16.09.2026

Reinforcement Learning



Motivation



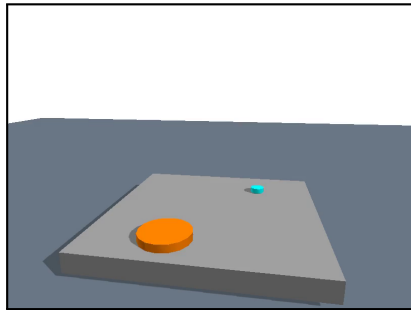
[Toussaint et al., “Combined Constrained Sampling and Reinforcement Learning for Robotic Manipulation”, CoRL 2026]

- *Only* the goal: Sparse reward, easy to specify, hard to train
- How to achieve the goal: Dense reward, hard to design, easier to train

Plan

Solve a manipulation task with RL

- Code 2 types of dense reward signals
- Code a strategy to handle sparse rewards
- RL training and evaluation
- Compare results



Plan

Stage 1 Installation, Setup, and Introduction (30 minutes)

- Clone the repository
- Native or docker-based installation
- Run a pre-trained policy
- Training and evaluation of a naive policy

Stage 2 Dense Rewards (30 minutes)

- `reward_fct_dense1()`
- `reward_fct_dense2()`
- Training and evaluation

Stage 3 Sparse Rewards (30 minutes)

- `getStartsGoals_curriculum()`
- Training and evaluation - easy mode
- Training and evaluation - hard mode

Getting to Know You Better

Audience Poll



<https://etc.ch/VNoT>

Stage 1: Installation and Setup

Installation and Setup

Please follow instructions in the repository README



https://github.com/LIS-TU-Berlin/rl_workshop_student

Overview of Code Repository

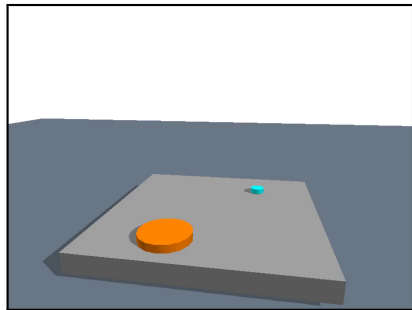
```
rl_workshop
|-- configs
|   |-- disc_push.yaml      # Base settings
|   |-- disc_push_task0.yaml # Settings for task 0
|   |-- disc_push_task1.yaml # Settings for task 1
|   |-- ...
|-- ...
|-- pyproject.toml          # Project settings and dependencies
|-- README.md               # Description and instructions
|-- rl_workshop
|   |-- ...
|   |-- problem_interface.py # Location for implementation
|   |-- sim_wrappers
|   |   |-- ...
|   |   |-- mujoco_gym.py    # Location for implementation
|   |   |-- ...
|-- runs                    # Training logs
|   |-- 260910-164238-disc_push-task1-sbTD3-seed100
|   |   |-- cfg.yaml
|   |   |-- eval_episodes.csv
|   |   |-- eval.mp4
|   |   |-- model.zip
|   |   |-- report.csv
|   |-- ...
```

[CONTINUED]

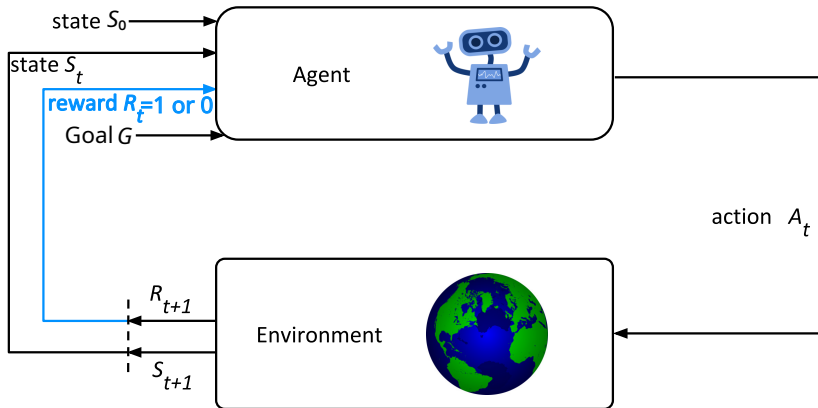
```
[CONTINUED]
|-- scenes
|   |-- disc_push.yaml      # Scene description
|-- scripts
|   |-- eval.py             # Evaluation script
|   |-- train.py            # Training script
|   |-- ...
|-- slides
|   |-- ...
|   |-- build
|   |   |-- ...
|   |   |-- slides.pdf
|-- tensorboard              # Tensorboard evaluation logs
|   |-- 260910-164238-disc_push-task1-sbTD3-seed100
|   |-- 260910-165339-disc_push-task2-sbTD3-seed100
|   |-- ...
|-- tests
|-- ...
```

The Environment

- Objective: Push the object to the goal
- Observation (see `observation_fct()` in `sim_wrappers/mujoco_gym.py`)
 - Finger position
 - Tracking error
 - Velocities
 - Finger-relative-to-object offset
 - Object-relative-to-goal offset
- Action (see `step()` in `sim_wrappers/mujoco_gym.py`)
 - Per-step ($\Delta x, \Delta y$) target $\rightarrow$ smooth spline trajectory
- Reward (to be implemented)
 - Dense or sparse



Reinforcement Learning



Reward: `reward_fct_sparse()`

Task 0: Sparse rewards, no special strategy, fixed goal

Success (sparse):

Whether goal was reached.

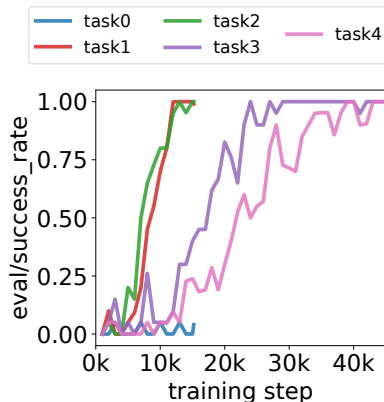
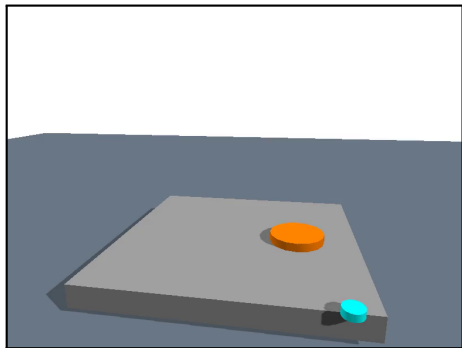
$$r_{\text{success}} = w_{\text{success}} \cdot \mathbb{1}[\sigma(f_t, g) \leq \varepsilon]$$

```
w_success = w_success,  
σ() = self.feat_distance(),  
f_t = feat,  
g = goal_feat,  
ε = goal_feat_eps,
```

Script: `sim_wrappers/mujoco_gym.py`

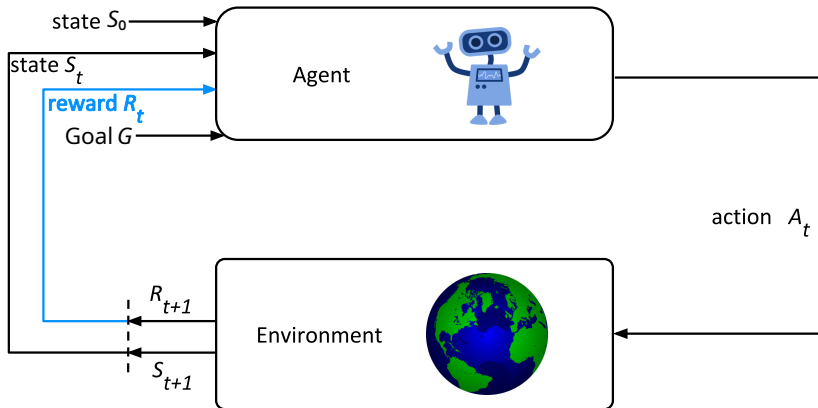
Reward: `reward_fct_sparse()`

Task 0: Sparse rewards, no special strategy, fixed goal



Stage 2: Dense Rewards

Reinforcement Learning



Reward: `reward_fct_dense1()`

Task 1: Dense reward, fixed goal

Success (sparse):

Whether goal was reached.

$$r_{\text{success}} = w_{\text{success}} \cdot \mathbb{1}[\sigma(f_t, g) \leq \varepsilon]$$

```
w_success = w_success,
σ() = self.feat_distance(),
f_t = feat,
g = goal_feat,
ε = goal_feat_eps,
```

Goal-progress (dense):

Change in obj-goal dist.

$$\begin{aligned} r_{\text{goal}} &= w_{\text{goal}} \cdot \phi_{\text{clip}}(\sigma(f_{t-1}, g), \sigma(f_t, g), d_{\text{clip}}) \\ &= w_{\text{goal}} \cdot \left(\max\left(1 - \frac{\sigma(f_t, g)}{d_{\text{clip}}}, 0\right) - \max\left(1 - \frac{\sigma(f_{t-1}, g)}{d_{\text{clip}}}, 0\right) \right) \end{aligned}$$

```
w_goal = w_goal,
φ_clip() = self.clipped_delta_reward(),
σ() = self.feat_distance(),
f_{t-1} = feat_prev,
g = goal_feat,
f_t = feat,
d_clip = reward_clip_dist (default 10 × ε)
```

Object-progress (dense):

Change in robot-obj dist.

$$r_{\text{object}} = w_{\text{object}} \cdot \phi_{\text{clip}}(\sigma_{\rho}(f_{t-1}), \sigma_{\rho}(f_t), \rho_{\text{clip}})$$

```
w_object = w_object,
φ_clip() = self.clipped_delta_reward(),
σ_ρ() = self.robot_obj_distance(),
f_{t-1} = feat_prev,
f_t = feat,
ρ_clip = reward_robot_obj_clip_dist
(default 10 × ε)
```

Total reward

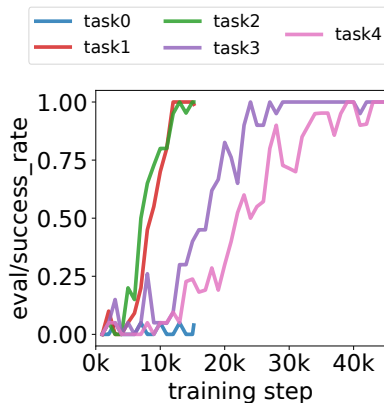
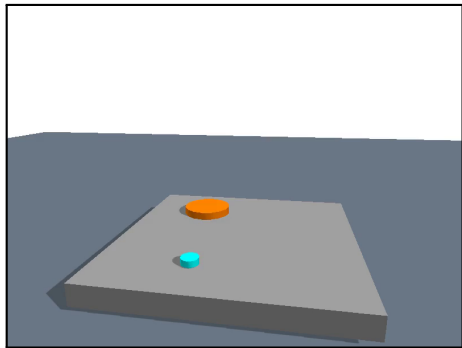
$$r = r_{\text{success}} + r_{\text{goal}} + r_{\text{object}}$$

Script: `sim_wrappers/mujoco_gym.py`

Color coding: `Code variables/functions`, `config variables`.

Reward: `reward_fct_dense1()`

Task 1: Dense reward, fixed goal



Reward: `reward_fct_dense2()`

Task 2: Dense reward with hitting component, fixed goal

Every reward component
from task 1 +

Hitting (dense):
Obj. velocity toward goal.

$$r_{\text{hit}} = w_{\text{hit}} \cdot \max(v_{\text{obj}} \cdot \hat{u}_{\text{goal}}, 0)$$

$$w_{\text{hit}} = w_{\text{hit}},$$
$$v_{\text{obj}} = \text{obj_vel[:, self.feat_dims]},$$
$$\hat{u}_{\text{goal}} = \frac{g - f_t}{\|g - f_t\|_2} = \frac{g - f_t}{\max(\sigma(f_t, g), 10^{-6})},$$

Total reward

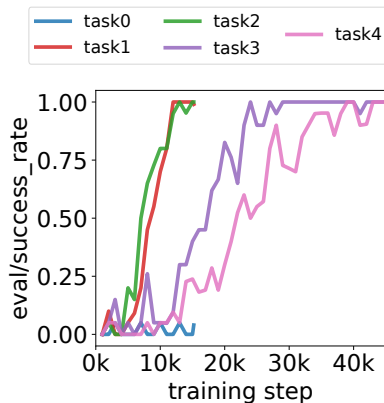
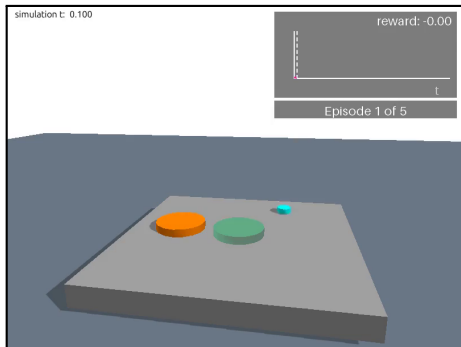
$$r = r_{\text{success}} + r_{\text{goal}} + r_{\text{object}} + r_{\text{hit}}$$

Script: `sim_wrappers/mujoco_gym.py`

Color coding: `Code variables/functions`, `config variables`.

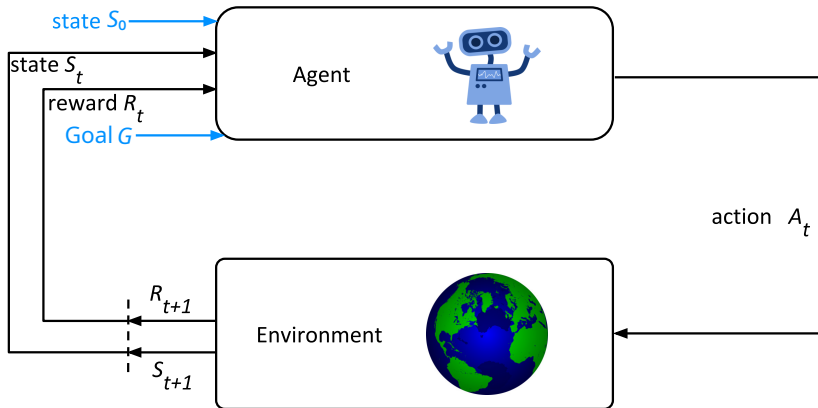
Reward: `reward_fct_dense2()`

Task 2: Dense reward with hitting component, fixed goal

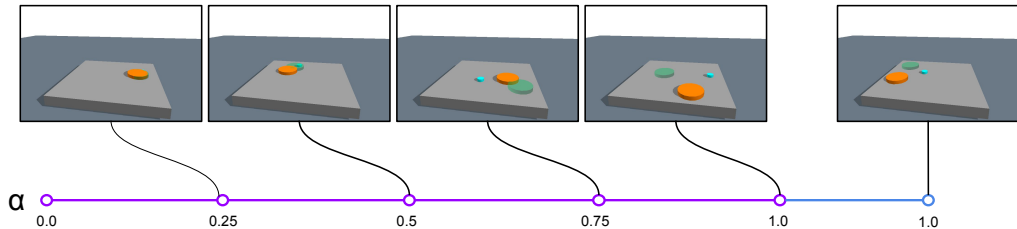


Stage 3: Sparse Rewards

Reinforcement Learning



Sampling and Curriculum



In each training step:

- Set α linearly based on training steps
- Sample two random *constrained* states $a, g \sim \mathcal{D}, s = \text{copy}(a)$
- $t \sim \mathcal{U}(0, \alpha)$
- $s[\text{obj_pos}] = t.a[\text{obj_pos}] + (1 - t).g[\text{obj_pos}]$

This is a simplified form of the **projected interpolation resets** and reset curriculum proposed in [Toussaint et al., "Combined Constrained Sampling and Reinforcement Learning for Robotic Manipulation", CoRL 2026]

Sampling and Curriculum

Algorithm 1: getStartsGoals_curriculum()

Input: n (number of pairs), $\alpha \in (0, 1]$ (curriculum ceiling), `final_evaluation` (bool)

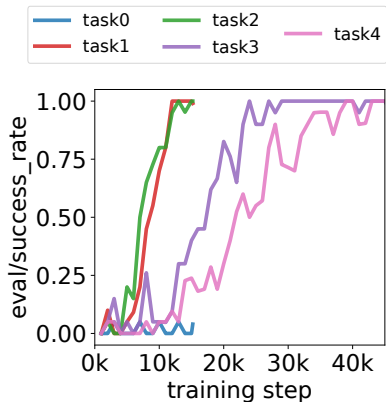
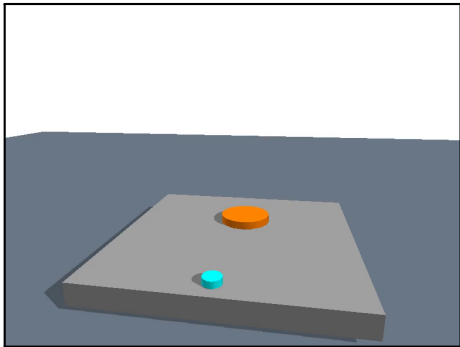
Output: Starts $\{q_i^{\text{start}}, r_i^{\text{start}}\}$, goals $\{q_i^{\text{goal}}, f_i^{\text{goal}}\}$, $i = 1, \dots, n$

```
...
for  $i = 1$  to  $n$  do
     $q^{\text{rand}}, q_i^{\text{goal}} \leftarrow$  sample random feasible configurations;
    ...
     $t \leftarrow 1$  if final_evaluation else  $t \sim \mathcal{U}(0, \alpha)$ ; //  $\alpha$  is linearly scaled by the curriculum,  $t$  depends on  $\alpha$ 
     $q_i^{\text{start}} \leftarrow q^{\text{rand}};$ 
     $q_i^{\text{start}}[\text{obj\_pos}] \leftarrow t q^{\text{rand}}[\text{obj\_pos}] + (1 - t) q_i^{\text{goal}}[\text{obj\_pos}];$  // interp start toward goal, capped by  $\alpha$ , obj_pos=[-7:-4]
    ...
return  $\{q_i^{\text{start}}\}, \{r_i^{\text{start}}\}, \{q_i^{\text{goal}}\}, \{f_i^{\text{goal}}\}$ 
```

This is a simplified form of the **projected interpolation resets** and reset curriculum proposed in [Toussaint et al., "Combined Constrained Sampling and Reinforcement Learning for Robotic Manipulation", CoRL 2026]

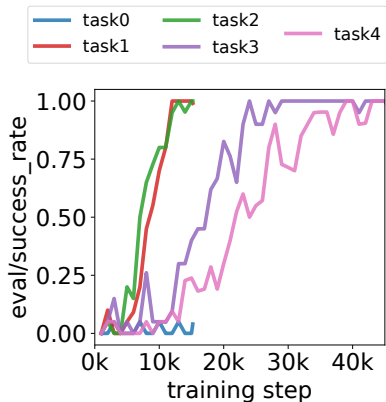
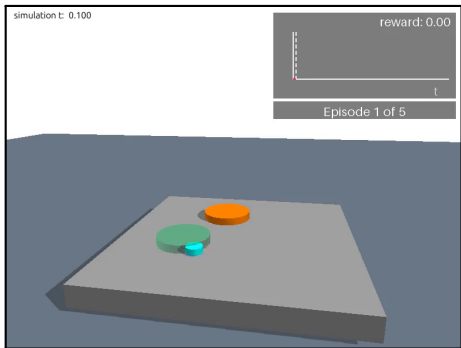
Sampling and Curriculum

Task 3: Sparse reward, easy mode (finger near object) , random goal

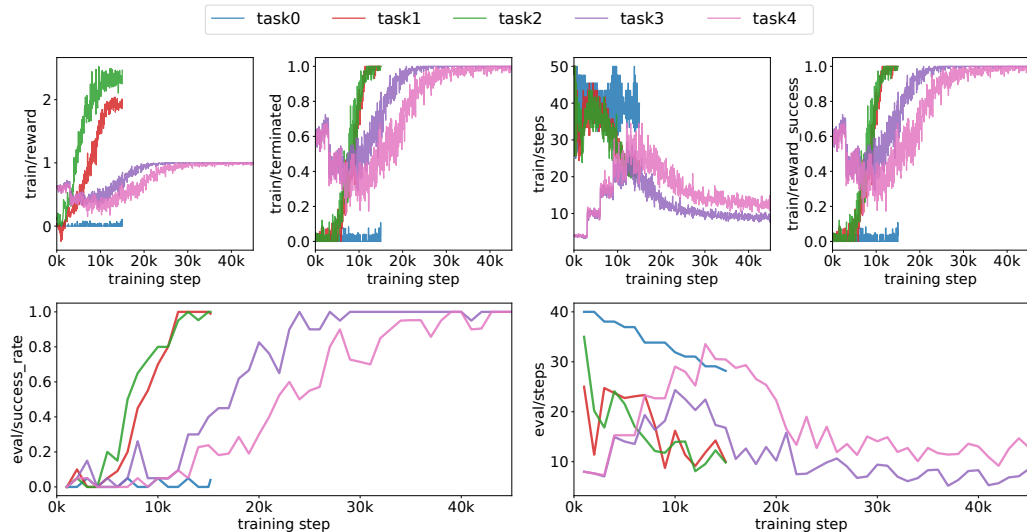


Sampling and Curriculum

Task 4: Sparse reward, hard mode (random finger), random goal



Results



Your Mission if You Choose to Accept it

- Setup [go to slide](#)
 - Cloning, installation
 - Run a pretrained policy
 - Train a naive policy
- Dense rewards
 - Implement `reward_fct_dense1()` [go to slide](#)
Train with `uv run scripts/train.py configs/disc_push_task1.yaml`
 - Implement the reward function `reward_fct_dense2()` [go to slide](#)
Train with `uv run scripts/train.py configs/disc_push_task2.yaml`
- Sparse rewards [go to slide](#)
 - Implement the sampling and curriculum strategy in `getStartsGoals_curriculum()` for sparse rewards
Train (easy mode) with `uv run scripts/train.py configs/disc_push_task3.yaml`
Train (hard mode) with `uv run scripts/train.py configs/disc_push_task4.yaml`

For details, refer to the repository README.

Resources

- **Reinforcement Learning: An Introduction:** Richard S. Sutton and Andrew G. Barto.
<http://incompleteideas.net/book/the-book-2nd.html>
- **Grokking Deep Reinforcement Learning:** Miguel Morales
<https://www.manning.com/books/grokking-deep-reinforcement-learning>
- **David Silver's RL Course:** lecture series covering RL foundations.
<https://www.davidsilver.uk/teaching/>
- **Spinning Up in Deep RL:** OpenAI's educational resource with theory + code.
<https://spinningup.openai.com/>
- **CleanRL:** Single-file, research-friendly implementations of RL algorithms.
<https://github.com/vwxyzjn/cleanrl>
- **Stable-Baselines3 (SB3):** Reliable PyTorch implementations of RL algorithms.
<https://stable-baselines3.readthedocs.io/>

env.close()

Thanks! Questions welcome.